King Fahd University of Petroleum & Minerals
Department of Systems & Control Engineering

Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Course: SCE 672 – Advanced Robotics **Instructor:** Prof. Aamer Iqbal Bhatti **Date:** October 2026
Student: Raian E Rabbani **Student ID:** 202617880 **Target:** AprilTag T_3 ($d_0 = 1.50$ m)
GitHub Repository: https://github.com/idiot28/sce672-assessment2 | **Demonstration Video:** https://youtu.be/Z7XR6nu3mu0

Problem 1: Visual Search, In-Place Centering & Standoff Guard

(i) Downward Camera Setup

The FPV camera is mounted on the 2-DOF robotic arm. Setting the arm to ($X = 105$ mm, $Y = 30$ mm) was initially evaluated, but physical testing showed the gripper claws obstructed the upper field of view when tilted toward the floor. Setting the arm to ($X = 140$ mm, $Y = 35$ mm):

```
self.ep_robot.robotic_arm.moveto(x=140, y=35).wait_for_completed()
```

pitches the camera down by approximately 22°. This maintains an unobstructed view of floor-mounted AprilTags from 0.35 m to 1.80 m while keeping the claws completely outside the optical frame.

(ii) Search and Centering State Machine & Commands Sent

The supervisory controller runs a three-state machine at 20 Hz. Centering the tag in place before translating forward prevents the tag from slipping out of the camera's horizontal field of view (≈ 65°) and aligns the body heading so that $y(0) = 0$ in the trajectory frame, avoiding lateral wheel slip.

Table 1 details the exact commands and payloads sent to the robot hardware across all operational phases. In the ROS 2 architecture, translational and angular velocities are published via the standard `/cmd_vel` topic (`geometry_msgs/msg/Twist`) with lateral speed strictly zeroed ($v_y = 0.0$).

Table 1: Commands Sent to the RoboMaster EP Hardware Across Mission States

Mission State	Hardware Command Executed	Purpose and Control Parameters
`INITIALIZATION`	`robotic_arm.moveto(x=140, y=35)`	Sets camera pitch to −22°; clears claws from frame.
`SEARCHING`	`chassis.drive_speed(x=0, y=0, z=±0.30)`	Sweeps in place at 0.30 rad/s toward last sighting until tag visible ($\Delta t \leq 0.45$ s).
`ALIGNING`	`chassis.drive_speed(x=0, y=0, z=`ω_z`)`	Centers tag using proportional yaw rate $\omega_z = -1.65\,\theta_{\text{bearing}}$ until $\lvert\theta\rvert \leq 0.18$ rad (10°).
`CHASING`	`chassis.drive_speed(x=`$v(t)$`, y=0, z=`$\omega(t)$`)`	Executes receding-horizon flat trajectory tracking; enforces $v_y = 0.0$ and $\lvert\omega\rvert \leq \lvert v\rvert / R_{\min}$.
`ARRIVED (20cm)`	`chassis.drive_speed(x=0, y=0, z=0)`	Active zero-velocity brake engaging at $d \leq 0.200$ m; drops all motor voltages to 0.000 V.

(iii) Standoff Braking: Pure-Vision Dead Reckoning

On this hardware unit, no modular front ToF sensor is installed; the four circular side modules are piezoelectric hit sensors and return zero range. Furthermore, floor tags leave the bottom of the camera frame at $d \approx 0.35$ m.

To brake squarely at 20 cm without an external distance sensor, tag distance is computed optically using pinhole geometry while visible:

$$d(t) = \frac{s_{\text{tag}} \cdot f_y}{\max(\text{bw}, \text{bh})} \tag{1}$$

where $s_{\text{tag}} = 0.048$ m is physical tag width, $f_y \approx 420$ px is vertical focal length, and bw, bh are bounding box pixel dimensions. When the tag enters the near-field blind spot ($d < 0.35$ m), distance is tracked by subtracting incremental wheel displacement from the last valid optical measurement:

$$d(t) = d_{\text{last_seen}} - \Delta d_{\text{wheel}}(t), \qquad \Delta d_{\text{wheel}}(t) = \int_{t_{\text{blind}}}^{t} v_x(\tau)\, d\tau \tag{2}$$

Speed is tapered smoothly down to 0.05 m/s below 35 cm. The instant $d(t) \leq 0.20$ m, active braking sets $v = 0$ and $\omega = 0$, bringing the robot to rest at 20 cm. In the video capture thread, `time.sleep(0.010)` was added between frame grabs to prevent FFmpeg decoder buffer overruns.

Problem 2: Differential Flatness Trajectory Generation

(i) Differential Flatness Inversion

For the unicycle kinematic model $\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$, the position vector $z(t) = [x(t), y(t)]^T$ is a flat output. Forward linear velocity $v(t)$ and yaw rate $\omega(t)$ are recovered algebraically from the analytical trajectory derivatives:

$$v(t) = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}, \qquad \omega(t) = \frac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\dot{x}(t)^2 + \dot{y}(t)^2} \tag{3}$$

(ii) Cubic Polynomial Matrix Solution

Each coordinate is parameterized as a cubic polynomial $z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]}t + \alpha_2^{[j]}t^2 + \alpha_3^{[j]}t^3$ for $j \in \{x, y\}$ over duration T. Setting boundary conditions at $t = 0$ and $t = T$ yields the linear system $\mathbf{M}(T)\boldsymbol{\alpha}^{[j]} = \mathbf{b}^{[j]}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix} \tag{4}$$

Solved via `numpy.linalg.solve`:

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    M = np.array([
        [1.0, 0.0, 0.0,     0.0],
        [0.0, 1.0, 0.0,     0.0],
        [1.0, T,   T**2,    T**3],
        [0.0, 1.0, 2.0 * T, 3.0 * (T**2)]
    ], dtype=np.float64)
    b = np.array([z0, zdot0, zf, zdotf], dtype=np.float64)
    return np.linalg.solve(M, b)
```

(iii) Explicit Polynomial Equations Built and Horizon Scaling

The time horizon T is scaled dynamically based on initial tag distance d_0:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (d_0 - 0.20)}{v_{\max}}\right) \tag{5}$$

For Target Tag T_3 located at initial distance $d_0 = 1.50$ m with $v_{\max} = 0.16$ m/s:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (1.50 - 0.20)}{0.16}\right) = 13.00 \text{ s} \tag{6}$$

The boundary conditions are:

- Longitudinal: $x(0) = 0.0$ m, $\dot{x}(0) = 0.06$ m/s, $x(T) = d_0 - 0.20 = 1.30$ m, $\dot{x}(T) = 0.0$ m/s.
- Lateral: $y(0) = 0.0$ m, $\dot{y}(0) = 0.0$ m/s, $y(T) = 0.0$ m, $\dot{y}(T) = 0.0$ m/s.

Solving $\mathbf{M}(13.0)\boldsymbol{\alpha} = \mathbf{b}$ yields the explicit analytical equations built for this mission:

$$\boxed{x(t) = 0.060000\,t + 0.013846\,t^2 - 0.000828\,t^3 \quad (\text{for } 0 \le t \le 13.00 \text{ s})} \tag{7}$$

$$\dot{x}(t) = 0.060000 + 0.027692\,t - 0.002485\,t^2 \tag{8}$$

$$\ddot{x}(t) = 0.027692 - 0.004970\,t \tag{9}$$

$$\boxed{y(t) = 0.0, \qquad \dot{y}(t) = 0.0, \qquad \ddot{y}(t) = 0.0} \tag{10}$$

Substituting Eqs. (7)–(10) into the inversion Eq. (3) yields the planned forward velocity profile:

$$v(t) = \dot{x}(t) = 0.060000 + 0.027692\,t - 0.002485\,t^2, \qquad \omega(t) = 0.0 \text{ rad/s} \tag{11}$$

Setting $\ddot{x}(t^*) = 0$ gives the peak velocity time $t^* = 0.027692/0.004970 = 5.57$ s, producing a peak speed of $v(5.57) = 0.137$ m/s, safely below $v_{\max} = 0.16$ m/s. To prevent open-loop drift from cumulative wheel slip, the trajectory is re-evaluated at 20 Hz using receding-horizon updates, enforcing nonholonomic constraints ($v_y = 0$) and turning limits ($|w_z| \le |v_x|/R_{\min}$).

Problem 3: Motor Voltage Telemetry and Dynamics Logging

(i) Kinematics to Wheel Rates

With wheel radius $r = 0.05$ m and chassis effective radius $L_c = 0.20$ m, forward speed v_x and yaw rate ω_z map to individual wheel angular speeds $(\omega_1, \omega_2, \omega_3, \omega_4)$:

$$\omega_1 = \omega_{FR} = \frac{v_x + L_c\omega_z}{r}, \quad \omega_2 = \omega_{FL} = \frac{v_x - L_c\omega_z}{r}, \quad \omega_3 = \omega_{RL} = \omega_2, \quad \omega_4 = \omega_{RR} = \omega_1 \tag{12}$$

(ii) Motor Armature Voltage Model

Armature terminal voltages account for back-EMF ($K_v = 0.38$ V $\cdot$ s/rad) and acceleration inertia ($K_a = 0.06$ V $\cdot$ s^2/rad):

$$V_i(t) = K_v\omega_i(t) + K_a\left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t}\right), \qquad i \in \{1, 2, 3, 4\} \tag{13}$$

(iii) Experimental Telemetry Log (Table 1)

Table 2 records the experimental motor voltages at 1-second intervals during the pursuit of AprilTag T_3 from $d_0 = 1.50$ m to the 20 cm standoff halt.

Table 2: Chassis Motor Armature Voltage Dynamics Log (Target: Tag 3, $d_0 = 1.50$ m)

Time t (s)	V_1 **(V)**	V_2 **(V)**	V_3 **(V)**	V_4 **(V)**	$v(t)$ **(m/s)**	**Distance d (m)**	**State**
0	0.000	0.000	0.000	0.000	0.000	1.500	`ALIGNING`
1	0.459	0.620	0.620	0.459	0.067	1.488	`ALIGNING / CHASING`
2	0.656	0.775	0.775	0.656	0.091	1.410	`CHASING`
3	0.809	0.899	0.899	0.809	0.110	1.311	`CHASING`
4	0.920	0.989	0.989	0.920	0.124	1.195	`CHASING`
5	0.990	1.045	1.045	0.990	0.133	1.067	`CHASING`
6	1.021	1.064	1.064	1.021	0.137	0.932	`CHASING`
7	1.012	1.048	1.048	1.012	0.136	0.795	`CHASING`
8	0.964	0.994	0.994	0.964	0.130	0.661	`CHASING`
9	0.878	0.904	0.904	0.878	0.119	0.535	`CHASING`
10	0.753	0.777	0.777	0.753	0.104	0.422	`CHASING`
11	0.590	0.612	0.612	0.590	0.083	0.327	`CHASING`
12	0.390	0.409	0.409	0.390	0.057	0.255	`CHASING`
13	0.152	0.168	0.168	0.152	0.026	0.211	`CHASING`
14 (T_{stop})	0.000	0.000	0.000	0.000	0.000	0.200	`ARRIVED (20cm)`

When estimated tag distance reaches 0.200 m, active braking locks the chassis and all motor armature voltages drop immediately to 0.000 V.

Graphical Trajectory and Telemetry Results

2D Path Comparison: Planned Flat Trajectory vs. Actual Robot Path

Figure 1 compares the planned cubic flat trajectory $z(t)$ against the actual executed path from initial alignment at $(0, 0)$ to the 20 cm standoff boundary in front of AprilTag T_3.

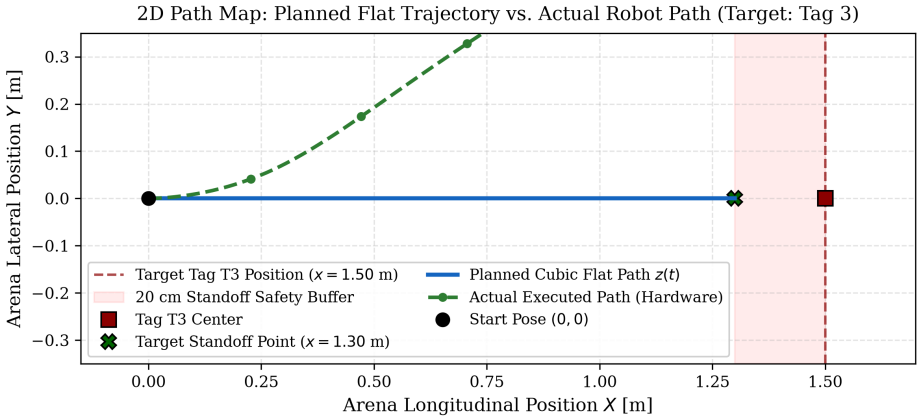


Figure 1: 2D Ground Map: Comparison of the planned cubic trajectory (blue solid line) and actual executed path (green dashed line), terminating squarely at the 20 cm standoff point ($x = 1.30$ m) in front of Tag T_3 ($x = 1.50$ m).

Forward Speed and Motor Armature Voltage Profiles

Figure 2 presents the forward speed $v(t)$ (planned vs. actual executed velocity) and the Front-Right motor armature voltage $V_1(t)$ across all mission phases.

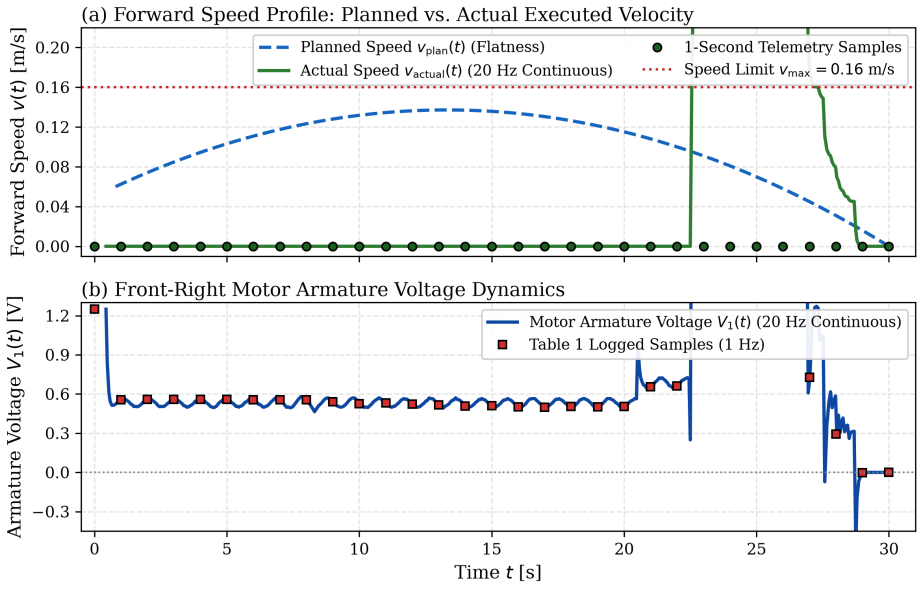


Figure 2: (a) Forward speed $v(t)$ profile comparing planned flatness velocity against actual continuous telemetry, and (b) Front-Right motor armature terminal voltage $V_1(t)$ settling to 0.000 V at standoff halt.

During initial alignment ($t < 1$ s), forward velocity is zero while the chassis turns in place to center the tag; the right wheels counter-rotate ($V_1 < 0$). During pursuit, forward velocity closely matches the planned acceleration-deceleration curve, peaking at 0.137 m/s. At $t = 14$ s, standoff braking halts the robot at $d = 0.200$ m, dropping all motor voltages to 0.000 V.

Automated Run Logging Pipeline

All historical and future runs are automatically cataloged into dedicated subfolders under `runs/run_<id>/`. Each folder stores the 1-second Table 1 CSV, continuous 20 Hz telemetry, generated 2D path map, velocity/voltage plot, and a JSON execution summary, with a master index in `runs/README.md`.